



NOWITNESS

Security Audit Report

Pondora Echo Contracts Security Audit

Date	February 27, 2026
Client	Pondora
Project	pondora-org/echo-contracts
Version	2.2.0

Contents

- Disclosure 1**
- Disclaimer 2**
- Scope 3**
- Assessment Overview 4**
- Assessment Components 5**
- Executive Summary 6**
- Code Base 8**
- Severity Classification 9**
- Finding Severity Ratings 10**
- Findings 11**
 - [M 301] Permissive Spending Requires Full Trust in Operator 12
 - [M 302] Inconsistent Consensus Key Types in Echo Validators 15
 - [L 201] Missing State Expiry Validation in Proof Minting 18
 - [I 101] Public Key vs NFT for Operator Minting 20
 - [I 102] Inefficient Storage of States 22

Disclosure

This document contains proprietary information belonging to No Witness Labs. Duplication, redistribution, or use, in whole or in part, in any form, requires prior written consent from No Witness Labs.

Nonetheless, both the customer **Pondora** and No Witness Labs are authorized to share this document with the public to demonstrate security compliance and transparency regarding the outcomes of the Protocol.

Disclaimer

This review reflects the state of the codebase at the time of assessment and should be regarded as a point-in-time evaluation. Any changes made after the assessment period fall outside the scope of this report.

Although every effort was made to identify potential risks and vulnerabilities, no security review can guarantee the discovery of all issues. The information and recommendations in this report are provided solely for technical purposes and must not be considered investment advice.

To strengthen security assurance, projects are encouraged to seek multiple independent reviews and to supplement audits with practices such as bug bounty programs.

The scope of this audit was limited to the code provided. It did not extend to compiler-level artifacts (e.g., generated UPLC code) or to areas beyond the explicitly reviewed codebase. Additional testing methodologies, including unit testing and property-based testing, were also outside the scope of this engagement.

Externally set parameters dependency

The protocol's design includes certain parameters that are set by external applications, such as the quorum size for validator consensus and the purpose identifier for validator operations. While the audit included an assessment of how these parameters are handled within the protocol, it did not evaluate the security implications of specific parameter choices or configurations. Users and integrators should exercise caution and conduct their own assessments when configuring these parameters for their specific use cases, as they can significantly influence the security and decentralization of the protocol.

Design intent dependency

Some payload signatures may be replayable by design. If future versions change the intended semantics (e.g., treating these signatures as explicit consent or requiring payload signatures to be context-specific), replayability could become security-relevant and should be re-evaluated.

Scope

In scope

- On-chain validators and supporting logic at the time of review.
- Datum/redeemer structures and on-chain invariants (value, units/rates, cross-phase checks).
- Use of reference UTxOs for configuration and script parameters.

Out of scope

- Compiler layer / UPLC output and toolchain internals.
- Creation of new unit tests or property-based tests.
- Off-chain libraries, front-end apps, wallet UX (incl. blind-signing), unless explicitly constrained on-chain.
- Third-party protocol internals (bridges/liquidity venues), centralized services, exchanges, oracles, external price feeds.
- Node operations, networking, mempool policy variance, ordering/MEV-like effects.

Assessment Overview

From **February 05, 2026** to **February 25, 2026**, **Pondora** engaged No Witness Labs to conduct a comprehensive security assessment of the **pondora-org/echo-contracts** protocol. The evaluation was performed in accordance with established industry best practices for code auditing and protocol analysis.

The audit followed a structured five-phase process:

- **Planning** – Defined the assessment scope and objectives, and aligned expectations with the client.
- **Discovery** – Acquired an understanding of the protocol through client discussions, documentation review, and preliminary analysis of architecture and design.
- **Manual Revision** – Conducted an in-depth review of the codebase to identify potential vulnerabilities, design weaknesses, and security risks.
- **Validation** – Verified client-provided fixes and obtained acknowledgments for resolved issues.
- **Reporting** – Documented all confirmed findings, assigned severity levels, and provided actionable recommendations for remediation.

The team also evaluated protocol efficiency and potential optimizations. Findings were prioritized by severity level.

Assessment Components

Manual revision

Our manual code auditing is focused on a wide range of attack vectors, including but not limited to:

- UTXO Value Size Spam (Token Dust Attack)
- Large Datum or Unbounded Protocol Datum
- EUTXO Concurrency DoS
- Unauthorized Data modification
- Multisig PK Attack
- Infinite Mint
- Incorrect Parameterized Scripts
- Other Redeemer
- Other Token Name
- Arbitrary UTXO Datum
- Unbounded protocol value
- Foreign UTXO tokens
- Double or Multiple satisfaction
- Locked Ada
- Locked non Ada values
- Missing UTXO authentication
- UTXO contention

Executive Summary

Echo is a consensus layer for coordinating membership and state on Cardano. It maintains an on-chain membership registry and a rotating consensus public key (FROST key) that can be used by external applications to require decentralized approval for sensitive off-chain actions. A coordinator bootstraps and orchestrates state updates, while Cardano validators enforce the membership and proof rules on-chain.

This report covers only the Echo on-chain validator set (“echo validators”): the membership state validator and the proof minting policy.

Contracts in scope

- **echo_members** (`validators/echo.ak`): Tracks membership state and the current consensus key.
 - Mints a single state token with an inline `EchoStateDatum` (`members_root`, `members_count`, `consensus_key`, `expires`).
 - Initializes state with a null membership root and coordinator key.
 - Updates state by appending new members, validating NFT ownership, and rotating the consensus key.
 - Allows burning old state tokens for cleanup.
 - Updates require a reference input to the previous state and a signature by the previous consensus key.
 - Members are appended only (never removed on-chain); off-chain consumers select acceptable snapshots.
- **echo_proof** (`validators/echo_proof.ak`): Mints a one-shot proof token consumable by external applications.
 - Uses a reference input to the current `echo_members` state.
 - Requires the consensus key in that state to sign the payload and validity range.
 - Enforces a minimum quorum via `members_count >= quorum`.

Membership and uniqueness Participation is gated by NFT ownership. A member is identified by holding the required NFT(s) at a unique smart account (Pond) address controlled by that member. Uniqueness and set size are enforced via an on-chain Merkle Patricia Trie (MPT): the on-chain root (`members_root`) and count

(`members_count`) prove a set of unique memberships, and the membership key in the MPT is the full NFT unit to prevent re-registration with the same NFT.

Integration model Consuming applications integrate by requiring an `echo_proof` that meets a defined minimum quorum and by ensuring that any payload associated with the proof cannot be replayed. This shifts the trust model away from a single operator by requiring a valid Echo proof before the application proceeds with a protected action.

Limitations and trust assumptions

- The chain does not verify that the consensus key corresponds to the full membership set; it proves that the previous key authorized the state transition that produced the current key.
- Threshold parameters for the FROST key are off-chain and must be published/verified by participants.
- The coordinator is responsible for bootstrapping and orchestrating updates.

Code Base

Repository

<https://github.com/pondora-org/echo-validators.git>

Branch

main

Commit

78a7296ee905ac7287e4018f6c5067ce132b510e

Files Audited

Files	SHA256 Checksum
validators/echo.ak	7e926355065b7bd25ae2e4ee1729514a9111b0c85f30be224d86aa25c2876b7d
validators/echo_proof.ak	53fb79c413fa366159fc54018c2234e9cdabe71d5d52adbaaa2a7f9de0e717a5
lib/echo/types.ak	a047941080c245a1663f30b3f81a3dc053c295eda639ad7e82884a90990f8da8
lib/echo/utils.ak	2d7d0b4b6e36dab1970cf4ed10a2f3d75947fe0dee5af0639c80dfb757b083af

Severity Classification

- **Critical:** Vulnerabilities that enable direct theft of assets, permanent loss of funds, or catastrophic protocol failure. These represent immediate and severe financial risk with straightforward exploitation paths.
- **High:** Vulnerabilities that can result in financial loss or significant harm to users or the protocol, though impact may be limited to specific functions or require certain conditions. Exploitation paths exist but may be more complex than Critical Vulnerabilities.
- **Medium:** Vulnerabilities that do not directly result in financial loss but may temporarily impair functionality or create exploitable conditions. These are typically limited to state manipulations and include issues such as front-running susceptibility or logic flaws that compromise transaction integrity.
- **Low:** Low-impact Vulnerabilities with limited attack vectors, minimal attacker incentive, or requiring highly specific scenarios. These typically do not result in financial loss or significant harm to users or the protocol.
- **Informational:** Findings without immediate security or financial implications. These include optimization opportunities, code quality improvements, inconsistencies in naming conventions, design suggestions, and gaps in documentation.

Impact–Likelihood Severity Matrix

	High likelihood	Medium likelihood	Low likelihood
High impact	C	H / M	M
Medium impact	H	M	M / L
Low impact	M	M / L	L

Finding Severity Ratings

The following table defines levels of severity and score range that are used throughout the document to assess vulnerability and risk impact

	Level	Severity	Count
	5	Critical	0
	4	High	0
	3	Medium	2
	2	Low	1
	1	Informational	2
Total Findings			5

Findings

ID	Title	Severity	Status
301	Permissive Spending Requires Full Trust in Operator	Medium	Resolved
302	Inconsistent Consensus Key Types in Echo Validators	Medium	Partially Resolved
201	Missing State Expiry Validation in Proof Minting	Low	Resolved
101	Public Key vs NFT for Operator Minting	Informational	Acknowledged
102	Inefficient Storage of States	Informational	Acknowledged

[M 301] Permissive Spending Requires Full Trust in Operator

	Level	Severity	Status
	3	Medium	Resolved

Category: Design Flaw

Affected File: `validators/echo.ak`

Description

The Echo protocol has asymmetric validation where state token minting requires strict validation, but spending is permissive. This allows tokens to leave script custody and enables any malicious UTxO containing the token to bypass all security guarantees.

Strict Minting Validation:

When minting/updating state tokens, the protocol enforces rigorous checks:

- Members prove NFT ownership at Pond addresses
- Merkle Patricia Trie ensures uniqueness
- Previous consensus key authorizes state transitions
- Membership count cryptographically proven

Loose Spending Validation:

However, spending state UTxOs only requires operator signature:

Code excerpt omitted at client request to avoid disclosing non-public source code.

This allows tokens to be spent from the script address and placed at any address with any datum, completely bypassing the strict validation guarantees established during minting.

Why This Matters:

The asymmetric validation means the strict minting guarantees can be completely bypassed. Once a token leaves script custody (through spending), it can be placed at any address with any datum. Since `echo_proof.ak` only validates the token exists (not the UTxO location or datum authenticity), any malicious UTxO containing the `echo_members` token can be used for proof minting.

Exploit Path:

1. Legitimate state creation with strict validation (NFT proofs, MPT, consensus signatures) - token at script address
2. Spend the token from script address (only requires operator signature - line 280)
3. Create malicious UTxO at any address with fake datum: `members_count: 1000000` (fake), `consensus_key: operator_key` (fake), `expires: infinity` (fake)
4. Mint proof using the malicious UTxO - bypasses all strict minting validation

Who Can Exploit:

- Operator (most obvious - can spend via line 280)
- Anyone who obtains the token through bugs, vulnerabilities, or future protocol changes
- Other scripts that receive the token (accidentally or maliciously)

Result: “Valid” proof with zero actual member consensus, completely bypassing all the expensive strict minting validation (NFT ownership, MPT computation, signature verification).

Impact

- Makes Strict Validation Pointless: Why require NFT ownership proofs, expensive MPT computation, and signature validation if tokens can leave script custody and be used with fake datums? All that expensive validation provides zero security.
- Defeats Security Model: The protocol’s strict minting rules establish security guarantees (proven members, verified NFTs, cryptographic uniqueness). The asymmetric validation allows these guarantees to be completely bypassed.
- Silent Failure for Applications: Consuming applications cannot detect malicious UTXOs on-chain. They assume `members_count >= quorum` means threshold consensus was achieved, but it could be fake data in an off-script UTXO.
- Broad Attack Surface: Not limited to operator - any scenario where the token leaves script custody (bugs, vulnerabilities, future changes, script interactions) creates the same risk.
- No On-Chain Detection: The malicious proof appears completely valid - proper token, well-formed datum, valid signature. External validators have no way to distinguish it from legitimate consensus.

Recommendation

Enforce token burn-only spending to maintain script custody and ensure all tokens used for proofs have been validated by the strict minting rules.

Option 1: Burn-Only Spending (Quick Fix)

- Modify the spending validation to only allow burning (negative minting) of state tokens
- This ensures tokens can only leave script custody through burning, maintaining the security guarantees of strict minting
- Conceptual implementation for burn-only spending:

Code excerpt omitted at client request to avoid disclosing non-public source code.

Option 2: Centralized Withdraw Logic (More Involved - Recommended)

- Implement a centralized withdraw logic that enforces burn-only spending for state tokens.
- Parameterize the validator with a `config_NFT` that holds the withdraw/script credentials (centralized protocol params).

- Export the burn logic to the withdraw script, and require that any spending of state tokens must call the withdraw logic to ensure proper validation.

Benefits:

- Prevents tokens from leaving script custody
- Still allows cleanup via burning
- Maintains security guarantees established during strict minting
- Eliminates the asymmetric validation vulnerability

Implementation Notes:

- Minimal breaking changes to existing functionality
- Maintains cleanup capability via burning
- Ensures tokens remain under script control
- Strict minting validation cannot be bypassed by spending

Why This Fix Works:

By restricting spending, tokens can only leave script custody through burning (negative minting). This ensures that any state token used in transactions has been validated by the strict minting rules and remains under script control. The asymmetric validation is eliminated - both minting AND spending now enforce proper validation.

Resolution

Enforced burn-only spending for state tokens to maintain script custody and ensure all tokens used for proofs have been validated by the strict minting rules

Code excerpt omitted at client request to avoid disclosing non-public source code.

Resolved in commit hash `2cb1092928e8c80fb304c7c3de5abb879a625e8f`

[M 302] Inconsistent Consensus Key Types in Echo Validators

	Level	Severity	Status
	3	Medium	Partially Resolved

Category: Cryptography

Affected Files:

- `validators/echo.ak`
- `validators/echo_proof.ak`

Description

The `echo.ak` validator expects the `consensus_key` to be a schnorr secp256k1 key, while the `echo_proof.ak` validator uses this key as an ed25519 key. This inconsistency can lead to a poisoned state where the validators are not able to verify signatures correctly.

In the `echo.ak` validator, the `verify_schnorr_secp256k1_signature` function is used to verify signatures, which expects a schnorr secp256k1 key. However, in the `echo_proof.ak` validator, the `verify_ed25519_signature` function is used, which expects an ed25519 key. This mismatch will cause signature verification to fail at either of the validators, depending on which one is being used.

Relevant code snippets:

In `validators/echo.ak`:

Code excerpt omitted at client request to avoid disclosing non-public source code.

Here the `previous_consensus_key` is expected to be a schnorr secp256k1 key, and the signature is verified using the `verify_schnorr_secp256k1_signature` function.

In `validators/echo_proof.ak`:

Code excerpt omitted at client request to avoid disclosing non-public source code.

Whereas in this validator, the `consensus_key` is expected to be an ed25519 key, and the signature is verified using the `verify_ed25519_signature` function.

Scenario 1

1. New state is initialized with a schnorr secp256k1 key as the `consensus_key`.
2. The state is updated, and the `echo.ak` validator verifies the signature using the `verify_schnorr_secp256k1_signature` function successfully.
3. Now a proof NFT is needed to be generated, and the `echo_proof.ak` validator attempts to verify the signature using the `verify_ed25519_signature` function.
4. The signature verification fails because the `consensus_key` is not an ed25519 key, leading to a poisoned state where the proof cannot be generated or verified.

Scenario 2

1. New state is initialized with an ed25519 key as the `consensus_key`.

2. A proof NFT can be generated successfully using the `echo_proof.ak` validator, as it expects an ed25519 key. But only if the `quorum` is set to 0.
3. However, when the state is updated, the `echo.ak` validator will fail to verify the signature using the `verify_schnorr_secp256k1_signature` function, as it expects a schnorr secp256k1 key. This again leads to a poisoned state where the state cannot be updated.

Scenario 3

1. New state is initialized with a malformed key that is neither a valid schnorr secp256k1 key nor a valid ed25519 key.
2. Both the `echo.ak` and `echo_proof.ak` validators will fail to verify signatures, leading to a poisoned state where neither state updates nor proof generation can occur.

Recommendation

Consensus Key Type Validation

Option 1: Standardize on Schnorr secp256k1 Keys

Ensure that both validators use the same type of key for the `consensus_key`. This can be achieved by standardizing on schnorr secp256k1 keys for both validators. This would involve updating the `echo_proof.ak` validator to use the `verify_schnorr_secp256k1_signature` function instead of the `verify_ed25519_signature` function.

Option 2: Use Both Key Types

Alternatively, if there is a specific reason to use both key types, include both the schnorr secp256k1 and ed25519 keys in the state datum, and ensure that each validator uses the appropriate key type for signature verification. This would involve modifying the state datum to include both keys and updating the validators to reference the correct key for their respective signature verification functions.

Key Correctness Validation

In the `echo.ak` validator, implement additional checks to ensure that the `consensus_key` provided is a valid schnorr secp256k1 key before adding it to the new state.

This can be done by signing the public key with the corresponding private key and verifying the signature using the `verify_schnorr_secp256k1_signature` function. If the verification fails, the state update should be rejected, preventing invalid keys from being used in the system.

Code excerpt omitted at client request to avoid disclosing non-public source code.

Resolution

- **Implemented:** Standardize on schnorr secp256k1 keys for the `consensus_key` in both validators to ensure consistent signature verification.
- **Not implemented:** Key correctness validation to ensure the `consensus_key` is a valid schnorr secp256k1 key before accepting it in state initialization or updates.

Resolved in commit hash `2cb1092928e8c80fb304c7c3de5abb879a625e8f`

[L 201] Missing State Expiry Validation in Proof Minting

	Level	Severity	Status
	2	Low	Resolved

Category: Consensus

Affected File: `validators/echo_proof.ak`

Description

The `echo_proof` validator mints proof tokens that attest to consensus approval for a given payload. While it validates that the referenced Echo state meets the required quorum, it does not verify that the state has not expired.

The `EchoStateDatum` includes an `expires` field that defines the temporal validity of the membership state, but this field is ignored during proof minting:

Code excerpt omitted at client request to avoid disclosing non-public source code.

Echo's security model relies on epoch-based state rotation to ensure membership snapshots remain current and reflect active NFT holders. The `expires` timestamp forces periodic state renewal where only current NFT holders are included. However, without expiry validation in proof minting, this mechanism is defeated.

Scenario illustrating the issue:

1	Day 1: Init state_v1, expires Day 30, 100 members
2	Day 15: Update state_v1, still expires Day 30, 120 members
3	Day 30: State EXPIRES
4	
5	Current behavior:
6	Day 31: Proof minting SUCCEEDS
7	Can use expired state_v1 indefinitely
8	
9	Expected behavior:
10	Day 31: Proof minting FAILS
11	Must create new state_v2 with current NFT holders

Impact

- Stale Consensus Authority: Proofs can be minted using a consensus key that no longer represents active NFT holders. Members who sold their NFTs and exited are still counted.
- Defeats Epoch-Based Rotation: The `expires` field is specifically designed to force periodic state renewal. Ignoring it completely defeats this core security mechanism, allowing states to remain valid indefinitely.

- Security Degradation Over Time: Days, weeks, or months can pass with members leaving while the expired state remains usable. The gap between claimed consensus and actual membership grows over time.
- Silent Acceptance: External applications consuming proofs cannot distinguish between live and expired state without additional off-chain verification. The on-chain proof appears valid even when using months-old expired states.

Recommendation

Add expiry validation to `echo_proof.ak` to ensure proofs can only be minted while the referenced state is temporally valid.

Modify `validators/echo_proof.ak` (line 61):

Code excerpt omitted at client request to avoid disclosing non-public source code.

This enforces that:

- The proof can only be minted while the referenced state is temporally valid
- State expiry is a hard boundary, preventing use of outdated membership snapshots
- External applications can trust that proofs reflect current consensus within the epoch window

Implementation Notes:

- No breaking changes to data structures or transaction patterns
- Coordinators must ensure they reference non-expired state when minting proofs
- Expired states will correctly fail validation after this fix

Resolution

Added `must_be_before` check for state expiry in proof minting

Resolved in commit hash `2cb1092928e8c80fb304c7c3de5abb879a625e8f`

[I 101] Public Key vs NFT for Operator Minting

	Level	Severity	Status
	1	Informational	Acknowledged

Category: Design

Affected File: `validators/echo.ak`

Description

In the `echo_members` validator, the `operator_mint_key` is currently defined as a `VerificationKeyHash`, which is a public key hash. This means that the operator minting process relies on a single public key for authentication.

While this approach is functional, it has some limitations in terms of security and flexibility.

Recommendation

Option 1: Replace the `operator_mint_key` with an NFT (Non-Fungible Token) instead of a public key. This would enhance the security and flexibility of the system in several ways:

1. Enhanced Security: An NFT can serve as a gatekeeper for the operator minting process. If the key associated with the NFT is compromised, the NFT can be moved to a new key, providing an additional layer of security. Note that if the key is compromised, the NFT associated with that key is also compromised, but the NFT can be transferred within a certain window of time.
2. Flexibility in Key Management: Using an NFT allows for more flexible key management and rotation. The operator can simply transfer the NFT to a new key if needed, without having to change the underlying logic of the validator.
3. Multi-Signature Wallets: The NFT can be placed in a multi-signature wallet, adding an extra layer of security and decentralization to the system.

Implementation Steps:

1. Add `operator_mint_nft_policy_id` and `operator_mint_nft_name` parameters to the `echo_members` validator to specify the NFT that will be used for operator minting.
2. Update the operator minting logic to check for an input containing the specified NFT.
3. Remove the `operator_mint_key` parameter from the validator, as it will no longer be needed.
4. Remove the code that checks for the `operator_mint_key` in the operator minting process.
5. Update any relevant code and documentation to reflect the change from a public key to an NFT for operator minting.

Option 2: If replacing the `operator_mint_key` with an NFT is not feasible, consider adding an additional NFT as a gatekeeper for the operator minting process.

Note: This option would not provide the same level of security as Option 1, but it would still enhance the flexibility of the system while allowing for efficient key management.

Implementation Steps:

1. Add `operator_mint_nft_policy_id` and `operator_mint_nft_name` parameters to the `echo_members` validator to specify the NFT that will be used for operator minting.
2. Update the operator minting logic to either check for the presence of the `operator_mint_key` or an input containing the specified NFT.
3. Update any relevant code and documentation to reflect the addition of the NFT as a gatekeeper for operator minting.

Resolution

Client is aware of the current design choice and is open to future modifications if needed.
No immediate action required.

[I 102] Inefficient Storage of States

	Level	Severity	Status
	1	Informational	Acknowledged

Category: Design optimization

Affected Files:

- `validators/echo.ak`
- `validators/echo_proof.ak`

Description

The Echo protocol is designed to preserve multiple states (snapshots) by storing them on-chain. Each state is represented as a datum `EchoStateDatum` that contains the following fields:

Code excerpt omitted at client request to avoid disclosing non-public source code.

The `members_root` field is a hash that represents the root of a Merkle Patricia Forestry (MPF) structure containing the members of the specified state (snapshot). The `members_count` field indicates the number of members in that state, while the `consensus_key` field holds the FROST key used for consensus. The `expires` field indicates the expiration time of the state.

Each state is stored in a separate UTXO, and a new token is minted for each state. This design allows for storing multiple states on-chain (practically, the protocol may have hundreds of states concurrently).

Problem

Since each UTXO requires a minimum amount of ADA to be locked, this design can become inefficient as the number of states increases, potentially requiring a significant amount of ADA to be locked in the UTXOs, leading to high costs and reduced scalability.

Recommendation

To optimize the design, we can store all states in a single MPF structure instead of having each state in a separate UTXO. This way, we can reduce the amount of ADA needed to lock in the UTXOs, improving cost efficiency and scalability, while preserving the ability to retrieve and verify states efficiently.

In this optimized design, we would have a single `EchoStateDatum` that contains the root of the MPF structure (which includes all states) and the `expires` field:

```
1 EchoStateDatum {
2   states_root: ByteArray, // root of the MPF structure that contains all EchoState
3   expires: Int,
4 }
```

 Aiken

Each individual state (snapshot) would be represented as an `EchoState` structure within the MPF:

```
1 EchoState {  
2     members_root: ByteArray,  
3     members_count: Int,  
4     consensus_key: FrostKey,  
5 }
```

Aiken

In this design, we would pass the `EchoState` in the redeemer when minting `echo_proof` token, and we would check the validity of the `EchoState` by verifying that it is included in the MPF structure stored on-chain in the `EchoStateDatum`. This way, we can ensure that the state is valid while significantly reducing the number of UTXOs needed to store the states, leading to improved efficiency and scalability.

Note: While this design optimization can significantly reduce the amount of ADA needed to lock in the UTXOs and improve efficiency, it introduces a tradeoff in terms of UTXO contention, as all updates would need to spend the same UTXO. This can potentially reduce throughput and complicate orchestration, especially if multiple updates need to be made concurrently. However, since each state update requires the operator's authorization, DoS attacks are not a concern, and concurrency can be managed by the operator, who can choose to update the state sequentially. If concurrency becomes a concern, we can mitigate this problem by having a few UTXOs that each holds a subset of the states, allowing for updates to be made in parallel, thereby reducing contention while still maintaining the benefits of storing states in fewer UTXOs.

Resolution

Client is aware of the current design choice and its implications, prioritizing simplicity and ease of interaction over cost efficiency. No changes were made.

End of Report



No Witness Labs — Confidential

Copyright © 2026